

LAPORAN PRAKTIKUM KOMPREHENSIF
TUGAS 2 OPERASI SYSTEM



Dosen Pengampu :
Triawan Adi Cahyanto M.kom

Disusun oleh:
HAFABI ARROHIM (2410651013) (A)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN 2025

Judul : Monitoring file menggunakan inotify di linux (WSL)

1. Tujuan praktikum

Tujuan dari praktikum ini adalah untuk memahami cara kerja file system monitoring pada sistem operasi Linux menggunakan bahasa C dan library inotify.

Program ini dibuat untuk memantau setiap aktivitas yang terjadi pada suatu direktori seperti pembuatan file, pengubahan isi file, dan penghapusan file.

2. Dasar teori

~sistem operasi dan file system

Sistem operasi menyediakan mekanisme untuk mengatur dan memantau file serta direktori. Pada Linux, hal ini dilakukan melalui system call.

~inotify

Inotify adalah fitur kernel Linux yang digunakan untuk memantau perubahan pada file atau direktori secara real-time. Dengan inotify, program dapat mendeteksi event seperti:

- IN_CREATE :file baru dibuat
- IN_MODIFY :file di ubah
- IN_DELETE :file di hapus

~system call dalam Bahasa C

Beberapa system call penting yang digunakan :

- Read() : membaca event dari file descriptor
- Write() : menulis log ke file teks
- Open() : membuka file untuk menulis log
- Close() : menutup file setelah selesai

~GCC (GNU compiler collection)

GCC digunakan untuk mengompilasi program C agar bisa dijalankan di Linux/WSL.

Contoh:

```
gcc file_monitor.c -o file_monitor
```

3. Deskripsi Program

Program ini bertugas memantau direktori /tmp/monitor_dir. Jika ada perubahan seperti file dibuat, diubah, atau dihapus, maka program akan mencatatnya ke file file_monitor_log.txt. Langkah-langkah jalannya program:

- Program dijalankan di terminal Linux menggunakan perintah:
./file_monitor
- Program mulai memantau folder /tmp/monitor_dir
- Saat file di folder itu dibuat, diubah, atau dihapus, terminal akan menampilkan pesan:
[17:26:32] File dibuat: test.txt
[17:29:35] File diubah: test.txt
[17:30:07] File dihapus: test.txt
- Semua log juga tersimpan otomatis di file file_monitor_log.txt

4. Flowchart program

1) Mulai



(2) Inisialisasi inotify instance



(3) Tentukan direktori yang akan dipantau (/tmp/monitor_dir)



(4) Baca event dari direktori



(5) Apakah ada perubahan file?

├─ Ya → Tulis log ke terminal dan ke file_monitor_log.txt

└─ Tidak → Kembali membaca event



(6) Ulangi terus sampai user menekan Ctrl + C



(7) Selesai

KODE SOURCE

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <sys/inotify.h>
#include <unistd.h>
#include <time.h>
#include <signal.h>
#include <string.h>
#include <errno.h>

#define BUF_LEN (1024 * (sizeof(struct inotify_event) + 16))

int fd, wd;

void handle_exit(int sig) {
    printf("\nProgram dihentikan.\n");
    inotify_rm_watch(fd, wd);
    close(fd);
    exit(0);
}

int main() {
    char buffer[BUF_LEN];
    time_t rawtime;
    struct tm *timeinfo;

    fd = inotify_init();
    if (fd < 0) {
        perror("inotify_init");
        return 1;
    }

    wd = inotify_add_watch(fd, "/tmp/monitor_dir", IN_CREATE | IN_MODIFY | IN_DELETE);
    if (wd < 0) {
        perror("inotify_add_watch");
        return 1;
    }

    signal(SIGINT, handle_exit);
    printf("Monitoring /tmp/monitor_dir ... Tekan Ctrl+C untuk berhenti.\n");

    while (1) {
        int length = read(fd, buffer, BUF_LEN);
        if (length < 0) {
            perror("read");
            break;
        }

        int i = 0;
        while (i < length) {
            struct inotify_event *event = (struct inotify_event *) &buffer[i];

            time(&rawtime);
            timeinfo = localtime(&rawtime);

            printf("[%02d:%02d:%02d] ",
                timeinfo->tm_hour, timeinfo->tm_min, timeinfo->tm_sec);

            if (event->mask & IN_CREATE)
                printf("File dibuat: %s\n", event->name);
            else if (event->mask & IN_MODIFY)
                printf("File diubah: %s\n", event->name);
            else if (event->mask & IN_DELETE)
```

OUTPUT YANG DI HASILKAN

```
faby@fabyar-biee: ~/praktik x + v
praktikum_file_monitor/file_monitor.c: In function 'main':
praktikum_file_monitor/file_monitor.c:45:9: error: expected '=', ',', ';', 'asm' or '__attribute__' before '_attribute_'
45 |     __attribute__((aligned(_alignof(struct inotify_event))));
    |     ^~~~~~
praktikum_file_monitor/file_monitor.c:45:9: warning: implicit declaration of function '_attribute_' [-Wimplicit-function-declaration]
praktikum_file_monitor/file_monitor.c:45:23: warning: implicit declaration of function 'aligned' [-Wimplicit-function-declaration]
45 |     __attribute__((aligned(_alignof(struct inotify_event))));
    |     ^~~~~~
praktikum_file_monitor/file_monitor.c:45:31: warning: implicit declaration of function '_alignof_' [-Wimplicit-function-declaration]
45 |     __attribute__((aligned(_alignof(struct inotify_event))));
    |     ^~~~~~
praktikum_file_monitor/file_monitor.c:45:41: error: expected expression before 'struct'
45 |     __attribute__((aligned(_alignof(struct inotify_event))));
    |     ^~~~~~
praktikum_file_monitor/file_monitor.c:50:27: error: 'buffer' undeclared (first use in this function); did you mean 'setbuffer'?
50 |     length = read(fd, buffer, sizeof(buffer));
    |                       ^~~~~~
praktikum_file_monitor/file_monitor.c:50:27: note: each undeclared identifier is reported only once for each function it appears in
faby@fabyar-biee:~$ cd praktikum_file_monitor
faby@fabyar-biee:~/praktikum_file_monitor$ nano file_monitor.c
faby@fabyar-biee:~/praktikum_file_monitor$ gcc file_monitor.c -o file_monitor
faby@fabyar-biee:~/praktikum_file_monitor$ ./file_monitor
Monitoring /tmp/monitor_dir ... Tekan Ctrl+C untuk berhenti.

[17:26:32] File dibuat: test.txt
[17:26:32] File diubah: test.txt
ls ~/praktikum_file_monitor

[17:29:35] File dibuat: test1.txt
[17:29:35] File diubah: test1.txt
b ^C
Program dihentikan.
faby@fabyar-biee:~/praktikum_file_monitor$ |
```

```
faby@fabyar-biee:~$ mkdir -p /tmp/monitor_dir
faby@fabyar-biee:~$ echo "test" > /tmp/monitor_dir/test.txt
faby@fabyar-biee:~$ mkdir -p /tmp/monitor_dir
faby@fabyar-biee:~$ cat ~/praktikum_file_monitor/file_monitor_log.txt
[2025-10-16 10:20:37] CREATED: test.txt
[2025-10-16 10:20:37] MODIFIED: test.txt
[2025-10-16 10:20:37] DELETED: test.txt
[2025-10-16 10:29:45] CREATED: test.txt
[2025-10-16 10:29:45] MODIFIED: test.txt
[2025-10-16 10:29:45] DELETED: test.txt
[2025-10-16 10:30:07] CREATED: test.txt
[2025-10-16 10:30:07] MODIFIED: test.txt
[2025-10-16 10:30:07] DELETED: test.txt
faby@fabyar-biee:~$ cat ~/praktikum_file_monitor/file_monitor_log.txt
[2025-10-16 10:20:37] CREATED: test.txt
[2025-10-16 10:20:37] MODIFIED: test.txt
[2025-10-16 10:20:37] DELETED: test.txt
[2025-10-16 10:29:45] CREATED: test.txt
[2025-10-16 10:29:45] MODIFIED: test.txt
[2025-10-16 10:29:45] DELETED: test.txt
[2025-10-16 10:30:07] CREATED: test.txt
[2025-10-16 10:30:07] MODIFIED: test.txt
[2025-10-16 10:30:07] DELETED: test.txt
faby@fabyar-biee:~$ |
```

5. Kesimpulan

Dari hasil praktikum dapat disimpulkan bahwa:

Program file monitor berhasil memantau aktivitas file secara real-time menggunakan inotify.

Setiap aktivitas file seperti pembuatan, pengubahan, dan penghapusan terdeteksi dan dicatat ke log.

Penggunaan system call di Linux dapat diintegrasikan dengan bahasa C untuk membuat aplikasi pemantau sistem.